

$\Delta \Sigma$ 変調器のDAC

ISHI会

フルデジタルで音声を操るためのロジック回路 5回目

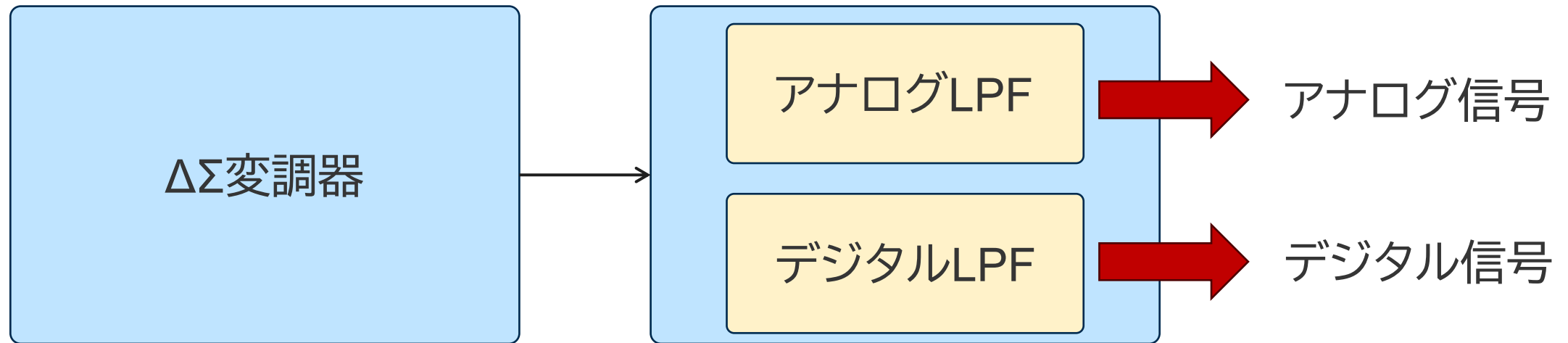
目次

- $\Delta\Sigma$ 変調を用いたADC
- $\Delta\Sigma$ 変調を用いたDAC

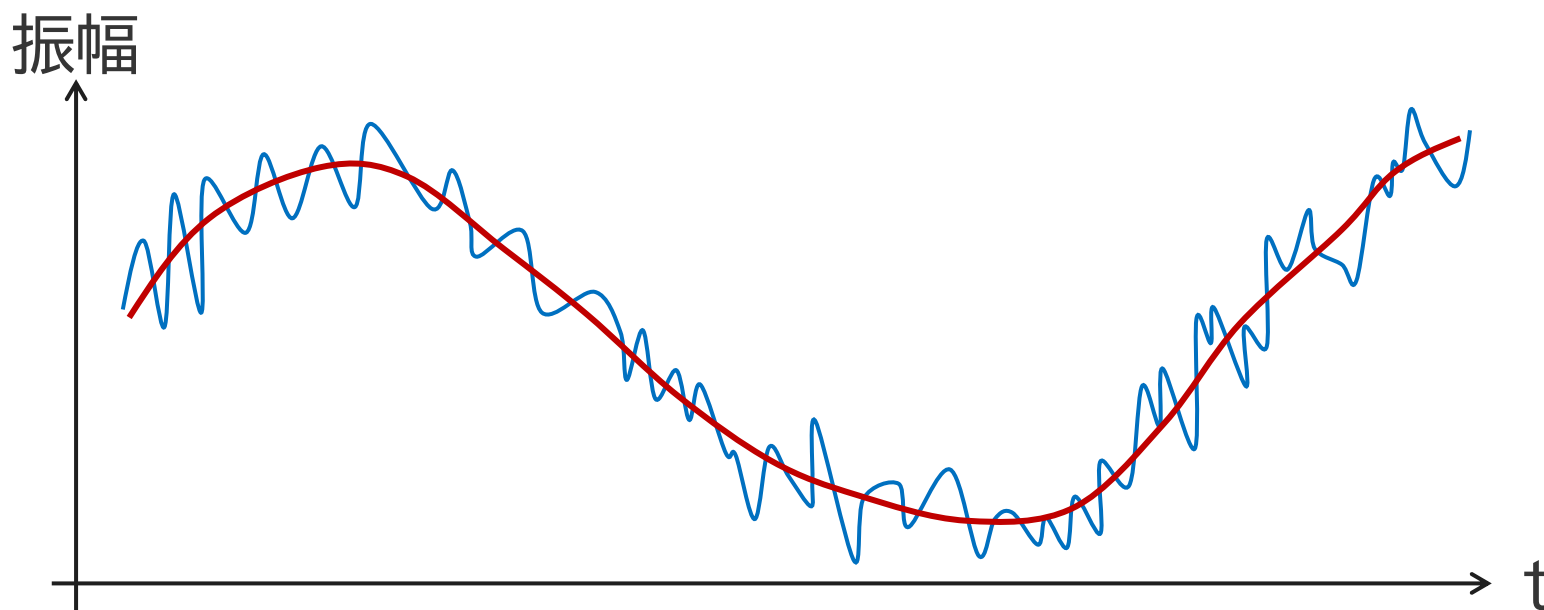
$\Delta\Sigma$ 変調を用いたADC

$\Delta\Sigma$ 変調器からデジタル信号を得る

- $\Delta\Sigma$ 変調器の2値パルス列からアナログ信号を得るにはLPFを通す必要があった
 - ✓ ローパスフィルタになるような処理をデジタルで実装出来ればADCとして動作



ノイジーな波形をなだらかにしたい



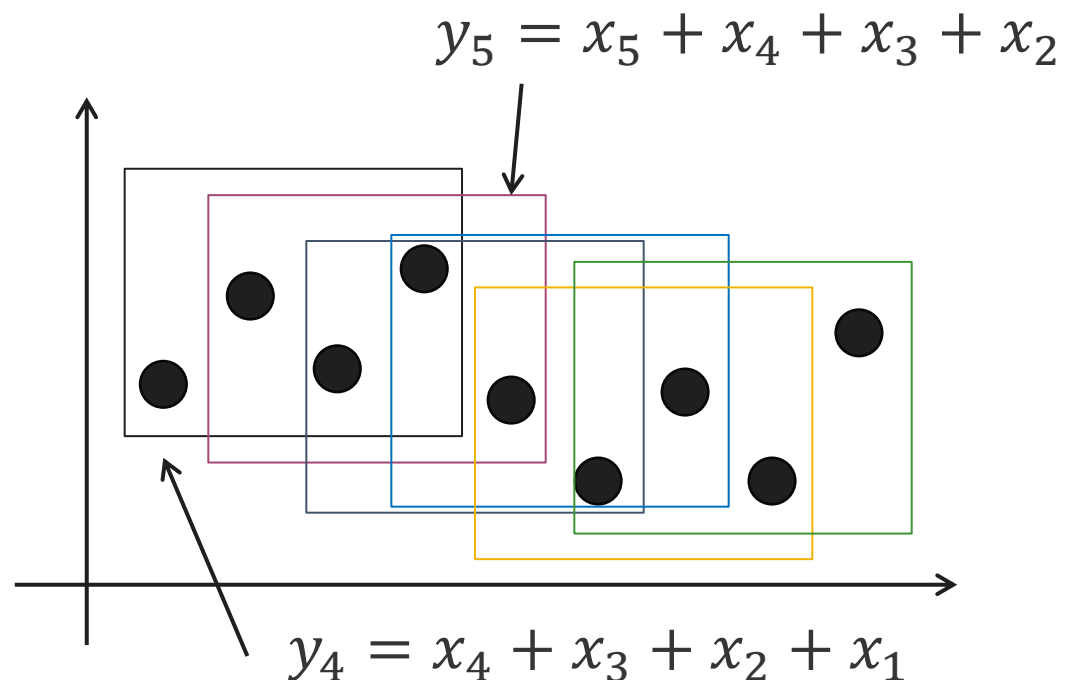
よくやるのは、区間を区切ってその区間の平均を求めながら、平均を求める区間を1点ずつずらす処理をします。

理系の人なら多分このような処理を一度は書いたこと(やったこと)があるのではないのでしょうか？

いわゆる移動平均です。

移動平均を考える

枠内の平均は？



y_5 と y_4 の差分を取ってみる。

$$y_5 - y_4 = x_5 - x_1$$

変形してみると、

$$y_5 = y_4 + x_5 - x_1$$

となるので、 n を使って書き直すと

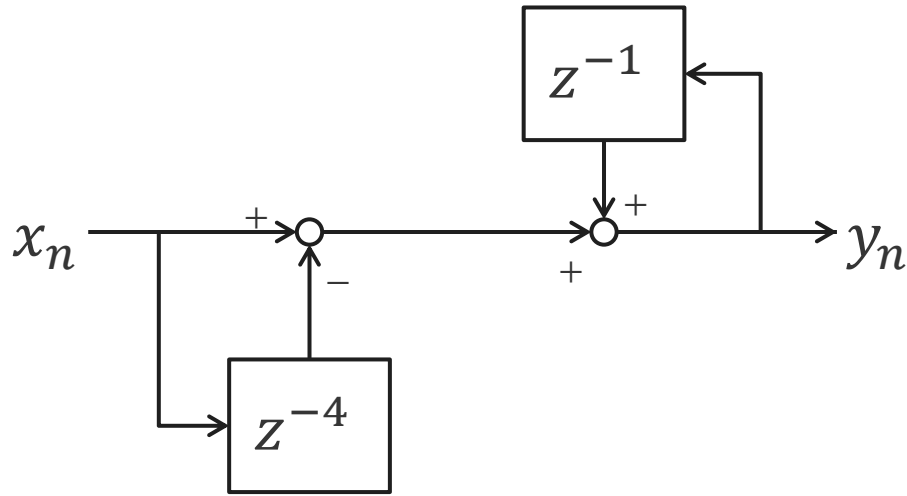
$$y_n = y_{n-1} + x_n - x_{n-4}$$

という感じで書くことができる。

※本当はデータ点数で割るが、ここでは一旦無視

移動平均の漸化式からブロック線図を起こしてみる

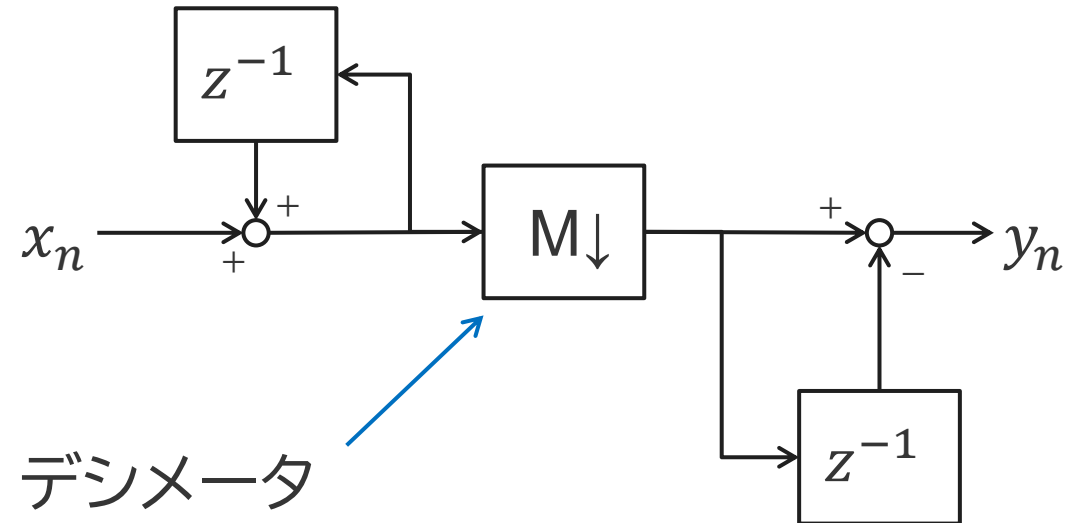
$$y_n = y_{n-1} + x_n - x_{n-4}$$



4点の移動平均はこのように書ける

移動平均をすると、なだらかなデータになる
 →高い周波数が落とされることになる
 →サンプリングレート下げても良い

先に積分をやるとサンプリングレートが下げられるので積分を先にやる。

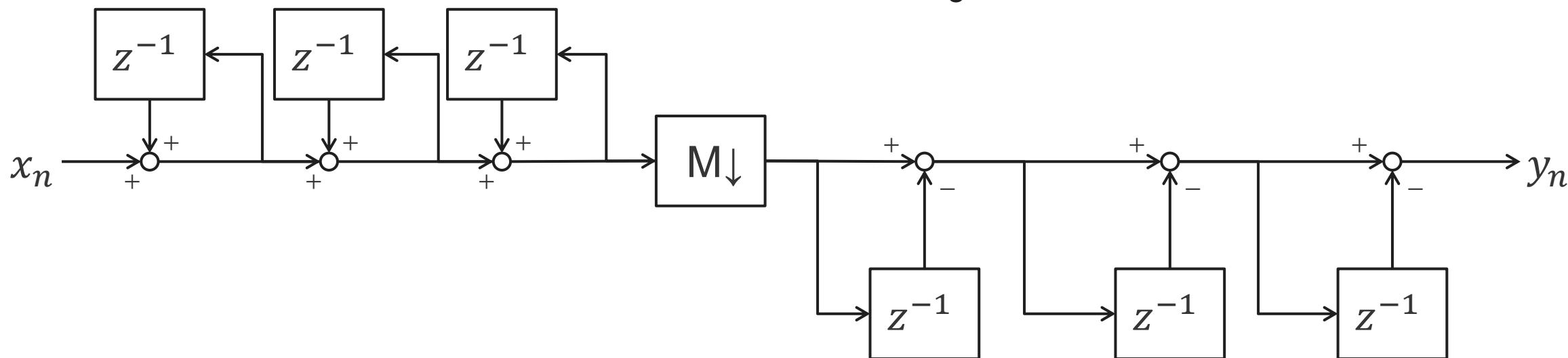


デシメータ
 データを間引く処理をする

CICフィルタ

移動平均フィルタは一般化するとCICフィルタと呼ばれる

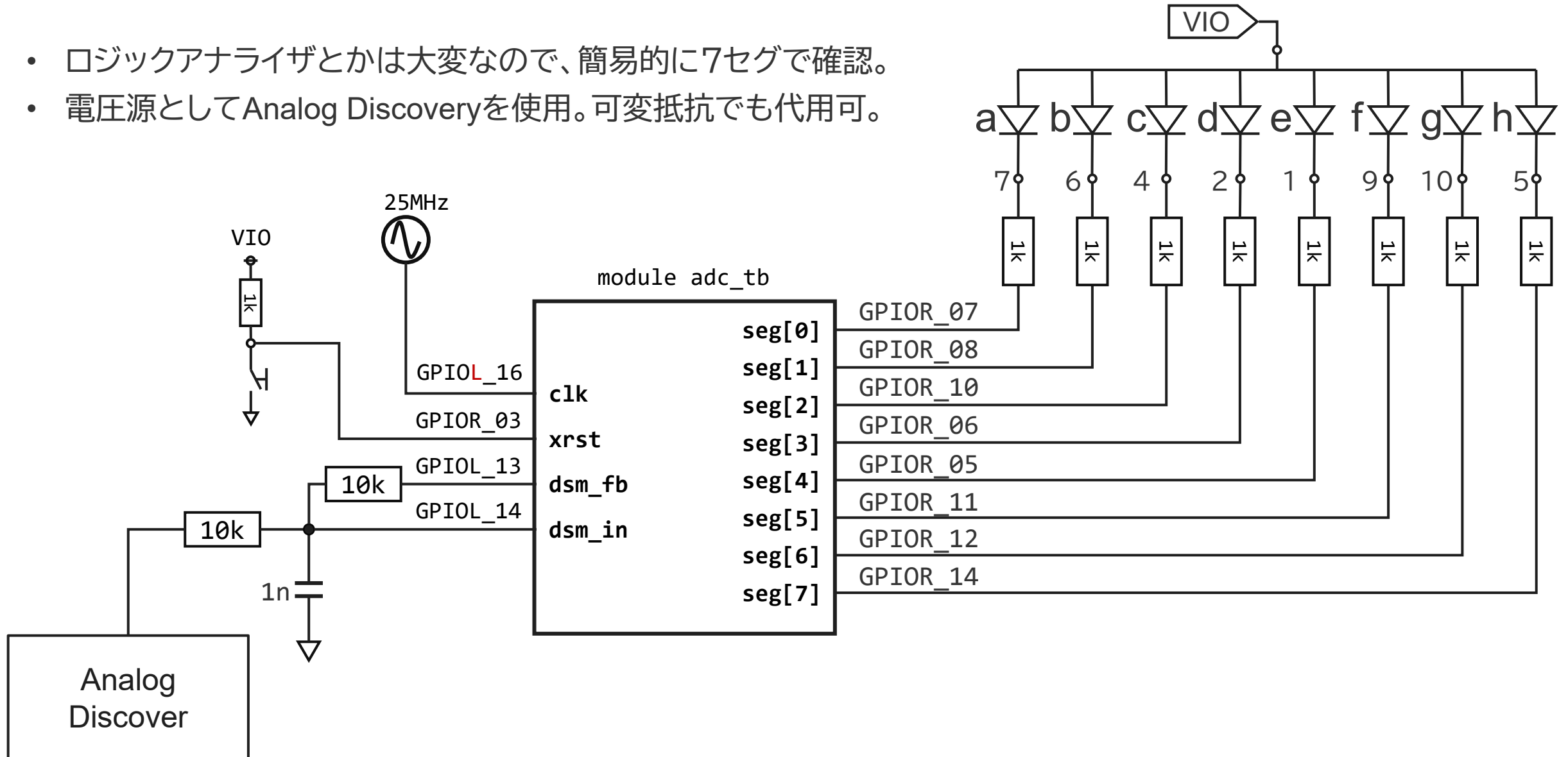
Cascaded Integrator-Comb



$\Delta\Sigma$ 変調器のフィルタとしては3段のCICフィルタがよく使われるので、この構成を実装してみる

ADC出力を7セグで確認

- ロジックアナライザとかは大変なので、簡易的に7セグで確認。
- 電圧源としてAnalog Discoveryを使用。可変抵抗でも代用可。



$\Delta\Sigma$ -ADCのVerilogコード

```
1 module dsm_adc(  
2     input wire clk,  
3     input wire xrst,  
4     input wire dsm_in,  
5     output reg dsm_fb,  
6     output reg pdm_out,  
7     output wire data_en,  
8     output wire [9:0] data_out);  
9  
10    always @(posedge clk or negedge xrst)  
11    begin  
12        if (!xrst)  
13        begin  
14            dsm_fb <= 1'b0;  
15            pdm_out <= 1'b0;  
16        end  
17        else  
18        begin  
19            dsm_fb <= ~dsm_in;  
20            pdm_out <= dsm_in;  
21        end  
22    end  
23  
24    wire [15:0] dout;  
25    assign data_out = dout[15:6];  
26  
27    cic_filter filt(  
28        .clk(clk),  
29        .xrst(xrst),  
30        .dsm_signal(pdm_out),  
31        .data_en(data_en),  
32        .data(dout));  
33  
34    endmodule  
35
```

この辺は前回と同じ($\Delta\Sigma$ 変調器)

出力は10ビットで出したかったので上位だけ取り出し

CICフィルタのインスタンス

CICフィルタのVerilogコード1

※この手のフィルタはAIが簡単に生成してくれるので、要件だけ決めてAIに投げちゃうのが楽だと思います。

```

1 module cic_filter(
2     input wire clk,
3     input wire xrst,
4     input wire dsm_signal, // 1ビットΔΣ入力 (0: -1 に対応, 1: +1 に対応)
5     output reg data_en,
6     output reg [15:0] data // 16ビット符号なし出力
7 );
8
9     /* 1. Input shaping (2-bit signed: +1 or -1) */
10    // 2の補数表現で、2'sb01 は +1、2'sb11 は -1 を表します。
11    wire signed [1:0] un_2b;
12    assign un_2b = (dsm_signal == 1'b0) ? 2'sb11 : 2'sb01;
13
14    // 内部レジスタ幅の26ビットに符号拡張
15    wire signed [25:0] un;
16    assign un = $signed(un_2b);
17
18    /* 2. Integrator part (26-bit) */
19    reg signed [25:0] acc1;
20    reg signed [25:0] acc2;
21    reg signed [25:0] acc3;
22
23    always @(posedge clk or negedge xrst) begin
24        if (!xrst) begin
25            acc1 <= 26'sd0;
26            acc2 <= 26'sd0;
27            acc3 <= 26'sd0;
28        end else begin
29            acc1 <= acc1 + un;
30            acc2 <= acc2 + acc1;
31            acc3 <= acc3 + acc2;
32        end
33    end
34
35    /* 3. Decimator part (Rate = 256) */
36    reg [7:0] counter;
37    reg dec_en;
38    always @(posedge clk or negedge xrst) begin
39        if (!xrst) begin
40            counter <= 8'd0;
41            dec_en <= 1'b0;
42        end else begin
43            counter <= counter + 8'd1;
44            if (counter == 8'd255) begin
45                dec_en <= 1'b1;
46            end else begin
47                dec_en <= 1'b0;
48            end
49        end
50    end

```

入力信号は0 or 1なので、-1 と 1に変換する

割とすぐに飽和するので、26ビットまで拡張(Geminiおすすめ)

積分器 3段CICなので積分器も3個

デシメータ 8ビットカウンタで256分の1に間引く
ここで間引くのではなく、データを取得するタイミング信号(dec_en)を作る

CICフィルタのVerilogコード2

```

52  /* 4. Differentiator (Comb) part (26-bit) */
53  reg signed [25:0] diff1_d;
54  reg signed [25:0] diff2_d;
55  reg signed [25:0] diff3_d;
56
57  reg signed [25:0] diff1;
58  reg signed [25:0] diff2;
59  reg signed [25:0] diff3;

```

微分器に使うレジスタの定義

```

60
61  always @(posedge clk or negedge xrst) begin
62      if (!xrst) begin
63          diff1_d <= 26'sd0;
64          diff2_d <= 26'sd0;
65          diff3_d <= 26'sd0;
66          diff1   <= 26'sd0;
67          diff2   <= 26'sd0;
68          diff3   <= 26'sd0;
69          data_en <= 1'b0;
70          data    <= 16'd0;

```

```

71  end else begin
72      if (dec_en == 1'b1) begin
73          // Comb段の差分演算
74          diff1   <= acc3 - diff1_d;
75          diff1_d <= acc3;
76
77          diff2   <= diff1 - diff2_d;
78          diff2_d <= diff1;
79
80          diff3   <= diff2 - diff3_d;
81          diff3_d <= diff2;

```

微分演算 3段CICフィルタなので3回微分する

```

82
83  // 5. 出力切り出し & 符号なし変換
84  // 入力が ±1 のとき、CICの最大出力振幅は  $\pm 2^{24} \approx \pm 16,777,216$  となります。
85  // 26ビット幅の中でデータが動くため、diff3[25]が符号（MSB）になります。
86  // 符号なし16ビットとして出力するため、[25:10]の16ビットを切り出し、MSBを反転します。
87  data    <= { ~diff3[25], diff3[24:10] };
88  data_en <= 1'b1;
89  end else begin
90      data_en <= 1'b0;

```

内部は26ビット符号ありで演算をしているので、16ビット符号なしに変換
デシメーションしているので、データ更新のタイミングを示すdata_en信号を生成

```

91  end
92  end
93
94
95  endmodule

```

$\Delta\Sigma$ -ADCのテストベンチVerilogコード

```

1 module adc_tb(
2   input wire clk,
3   input wire xrst,
4   input wire dsm_in,
5   output wire dsm_fb,
6   output wire [7:0] seg);
7
8   wire [9:0] data_out;
9   wire data_en;
10  reg [3:0] code;
11
12  dsm_adc uut(
13    .clk(clk),
14    .xrst(xrst),
15    .dsm_in(dsm_in),
16    .dsm_fb(dsm_fb),
17    .pdm_out(),
18    .data_en(data_en),
19    .data_out(data_out));
20
21  seg_disp code_to_seg(
22    .code(code),
23    .seg(seg));
24
25  always @(posedge clk or negedge xrst)
26  begin
27    if (!xrst)
28    begin
29      code <= 4'd0;
30    end
31    else
32    begin
33      if (data_en == 1'b1)
34        code <= data_out[9:6];
35    end
36  end
37
38 endmodule

```

$\Delta\Sigma$ -ADCのインスタンス。pdm_outは使わないので空欄

4ビットから7セグLEDに変換するモジュール(第2回で作成したもの)

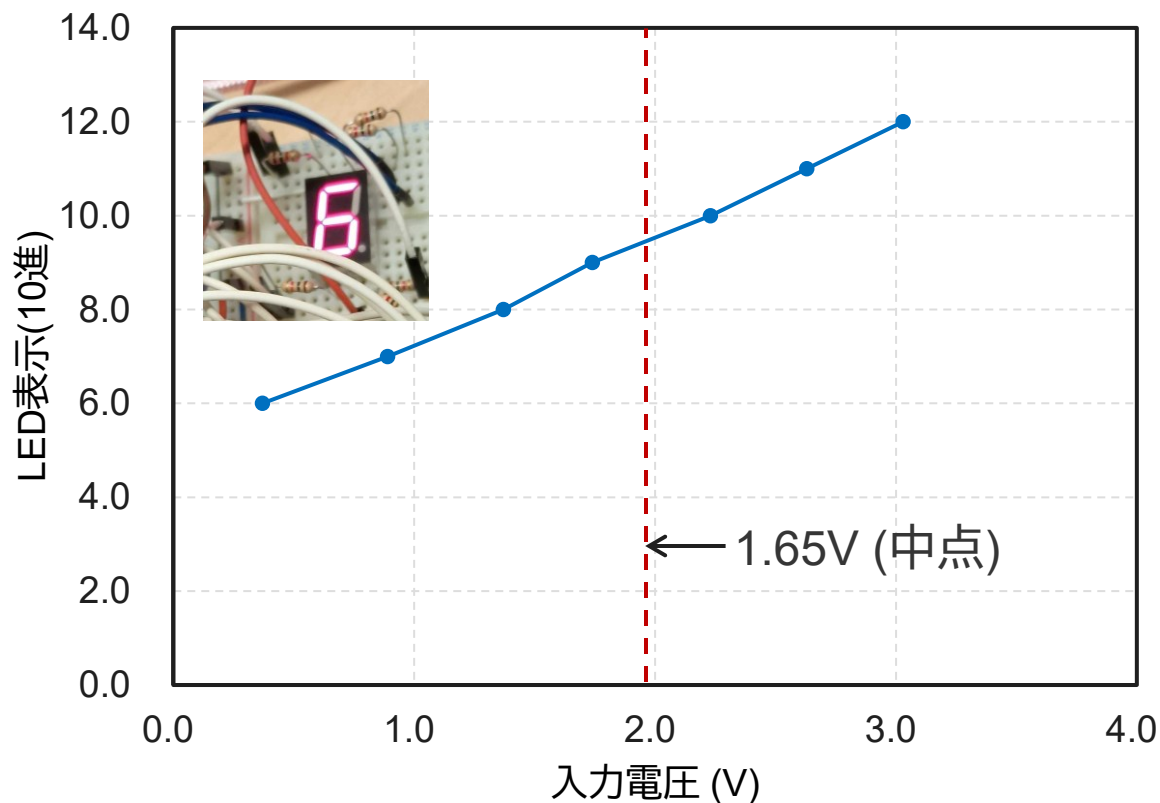
ADCのデータを7セグ用に変換。上位4ビットだけを7セグデコーダモジュールに接続
そのまま繋いでもいいが、data_enが1のタイミングでレジスタを更新

ピンアサイン

GPIO : Instance View

Instance	Package Pin	Resource	I/O Bank	Alt Conn	Features	Clock Region	Pad
clk	C2	GPIOL_16	1B	GCLK	None	L1	GPIOL_16_CLK2
dsm_fb	F1	GPIOL_13	1A	GCTRL	None	L0	GPIOL_13_CTRL1
dsm_in	E2	GPIOL_14	1A	GCLK	None	L0	GPIOL_14_CLK0
seg[0]	C7	GPIOR_07	2A	None	None	R1	GPIOR_07
seg[1]	A8	GPIOR_08	2A	None	None	R1	GPIOR_08
seg[2]	A9	GPIOR_10	2A	None	None	R1	GPIOR_10
seg[3]	C6	GPIOR_06	2A	None	None	R1	GPIOR_06
seg[4]	B6	GPIOR_05	2A	None	None	R1	GPIOR_05
seg[5]	B8	GPIOR_11	2A	None	None	R1	GPIOR_11
seg[6]	C8	GPIOR_12	2A	None	None	R1	GPIOR_12
seg[7]	B9	GPIOR_14	2A	None	None	R1	GPIOR_14
xrst	A6	GPIOR_03	2A	None	None	R1	GPIOR_03

$\Delta \Sigma$ -ADCの実験結果



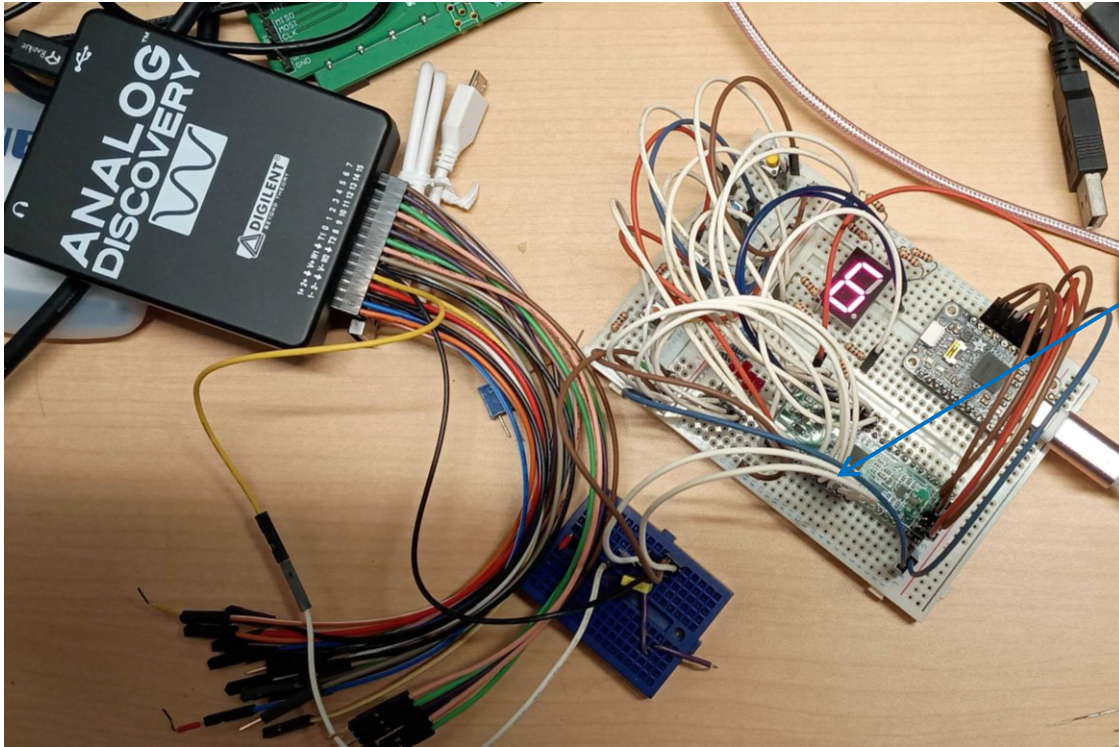
- 表示される数値は入力電圧に対して線形に変化
- フルスケール入力に対して、表示はフルスケールしない
 - CICの結果にスケールリングが必要
- 上方向にオフセットが見られる
 - オフセット補正が必要



性能はあまり良くないが、ADCとしての動作はしているように見える。

なんかイマイチな理由

GNDが弱いようでAnalog DiscoveryのGNDのつなぐ場所を変えると値が変わる...



この2本のケーブルがdsm_inとdsm_fb
→25MHzのパルスが流れている。
→めちゃくちゃノイズを出すし、GNDが揺れる



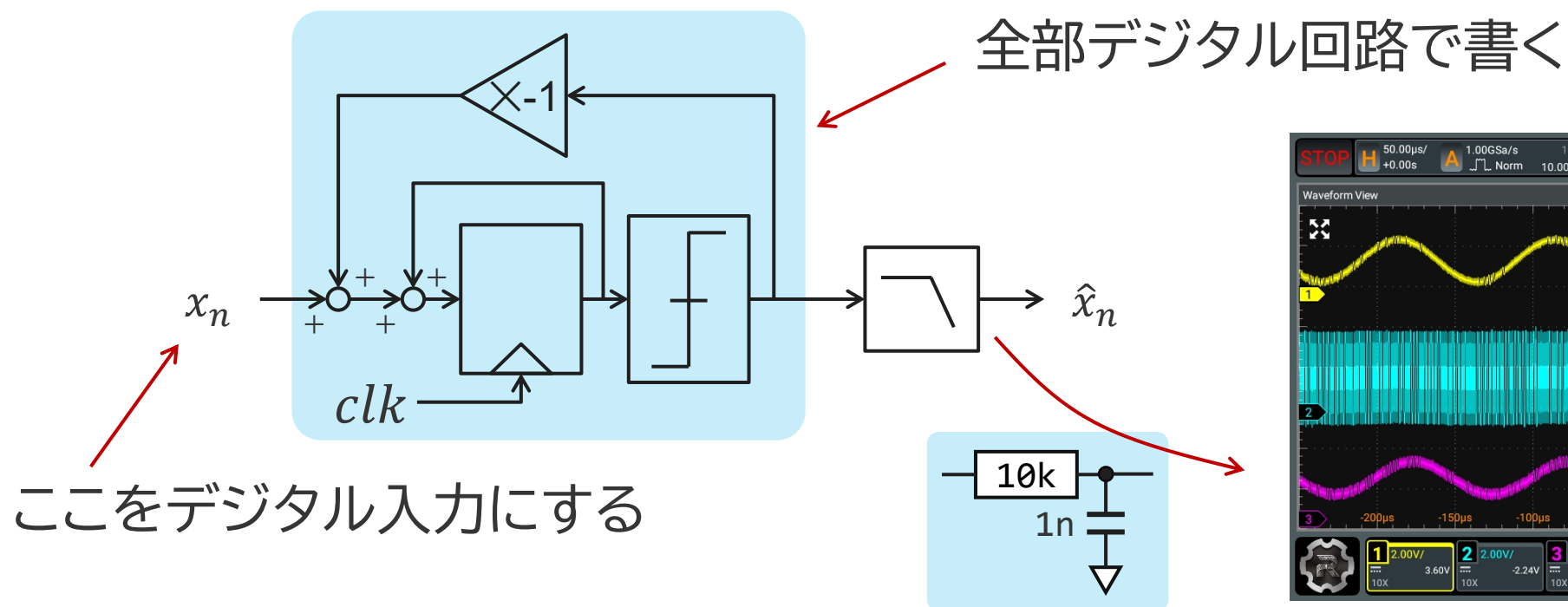
- クロックをもっと遅くする
- ちゃんとした基板を起こす

$\Delta \Sigma$ 変調を用いたDAC

$\Delta\Sigma$ 変調器からアナログ信号を得る

- $\Delta\Sigma$ 変調した2値の信号からアナログ信号を得るにはRCローパスフィルタがあれば実現できた
- これまで、 $\Delta\Sigma$ 変調器の入力はアナログ信号だった
- デジタル信号を入力する $\Delta\Sigma$ 変調器はどのように構成するのか？

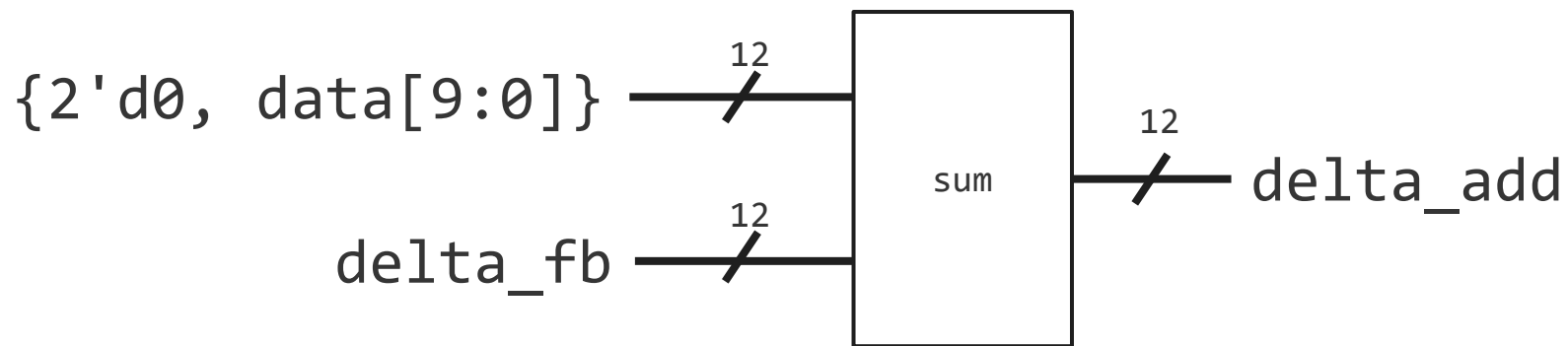
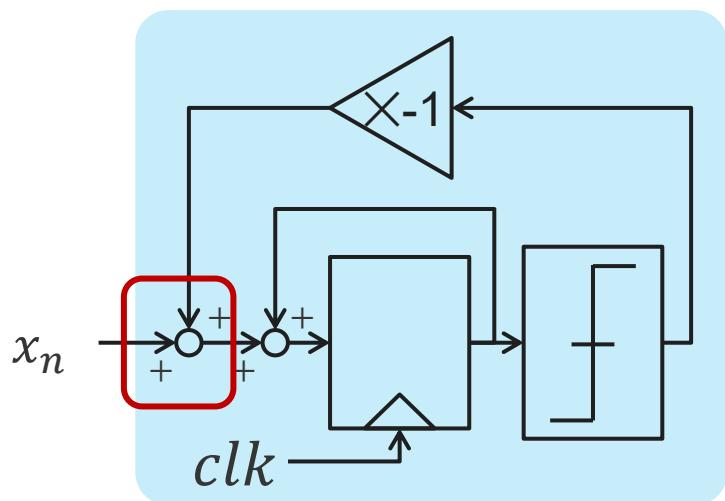
$\Delta\Sigma$ 変調器



デジタル入力の $\Delta \Sigma$ 変調器1

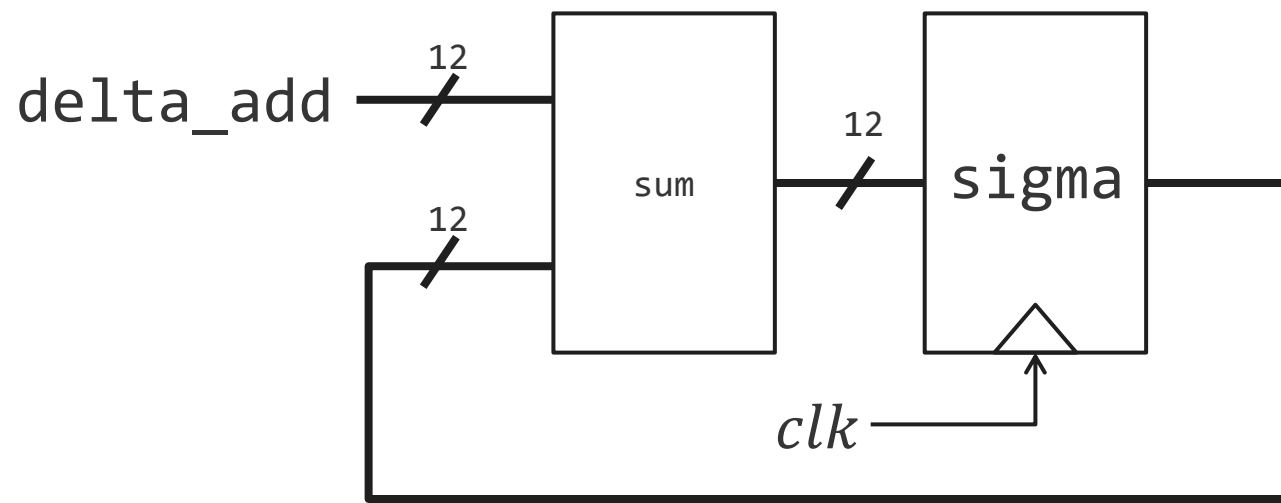
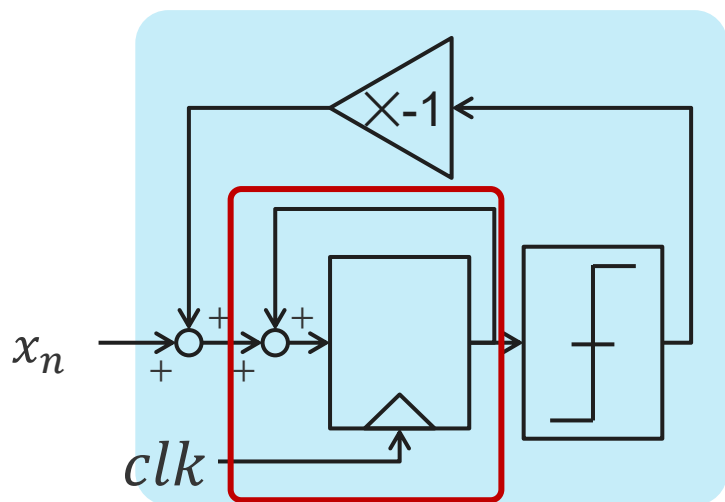
ブロック図をVerilogで記述するためにブロック図をそれぞれのブロックに分解する

- 入みにフルレンジが入ると積分器が飽和する可能性があるためビット拡張する。
 - このビット拡張は差分の計算を簡単にする意図もある→後述



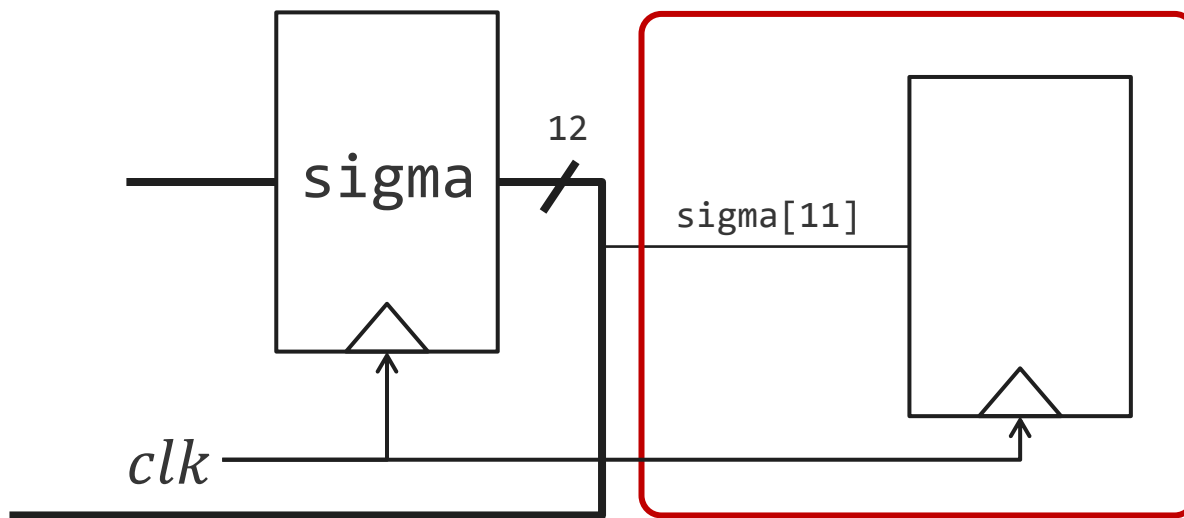
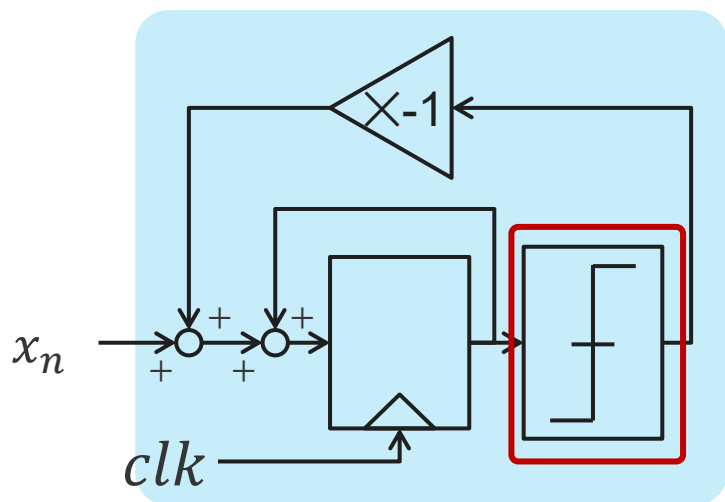
デジタル入力の $\Delta \Sigma$ 変調器2

- 差分は積分器に入力する。
- 積分器は加算器とレジスタで構成される。



デジタル入力の $\Delta \Sigma$ 変調器3

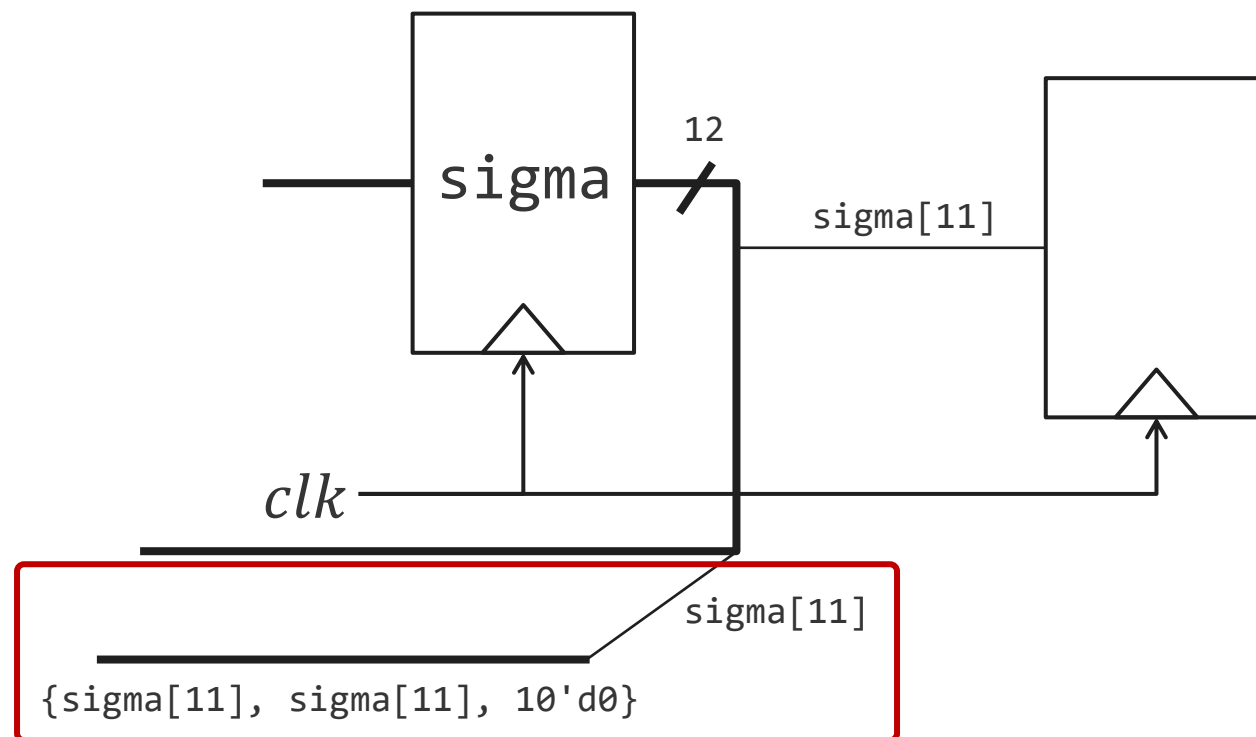
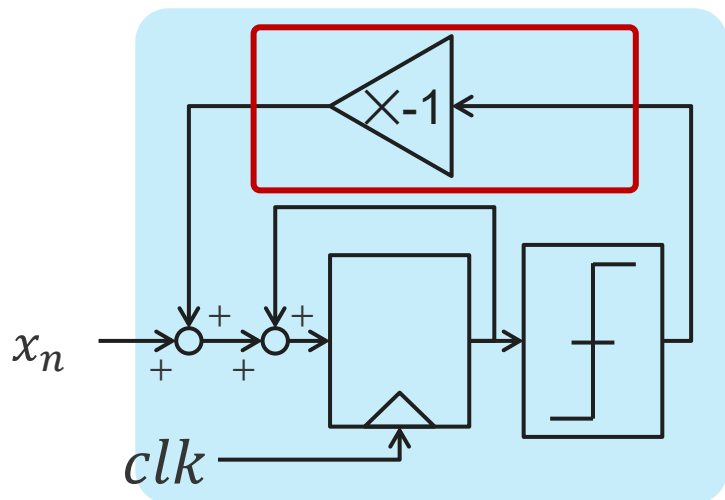
- コンパレータは積分器出力の最上位ビットを見る
- 最上位ビットが1であれば、フルレンジの半分より上なので1を出力する。
- 最上位ビットが0であれば、フルレンジの半分未満なので、0を出力する。



デジタル入力の $\Delta\Sigma$ 変調器4

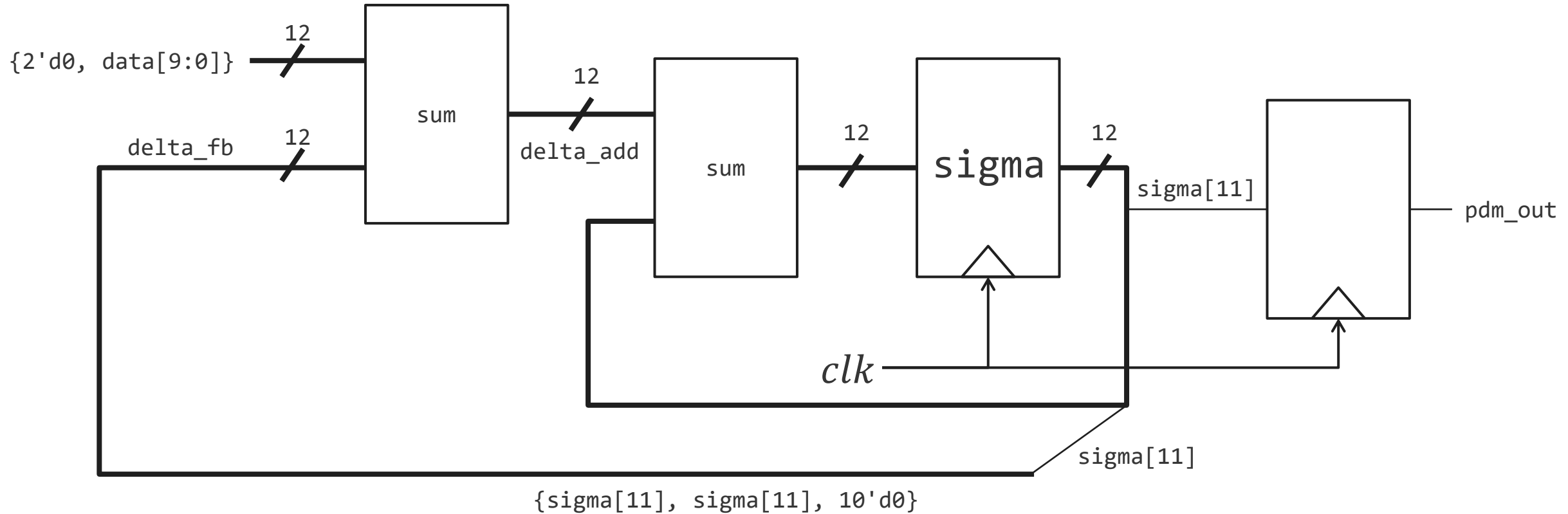
- 2値化した値が1だったとき、 Δ は負になるべきなので2の補数表現で**負の値**に相当する最上位ビットが**1となる値をフィードバック**する。
- 2値化した値が0だったとき、 Δ は正になるべきなので2の補数表現で**正の値**に相当する最上位ビットが**0となる値をフィードバック**する
 - ✓ 2ビットのビット拡張があるので、sigma[11]で2ビット分拡張する

10進	2進
1	01
0	00
-1	11
-2	10



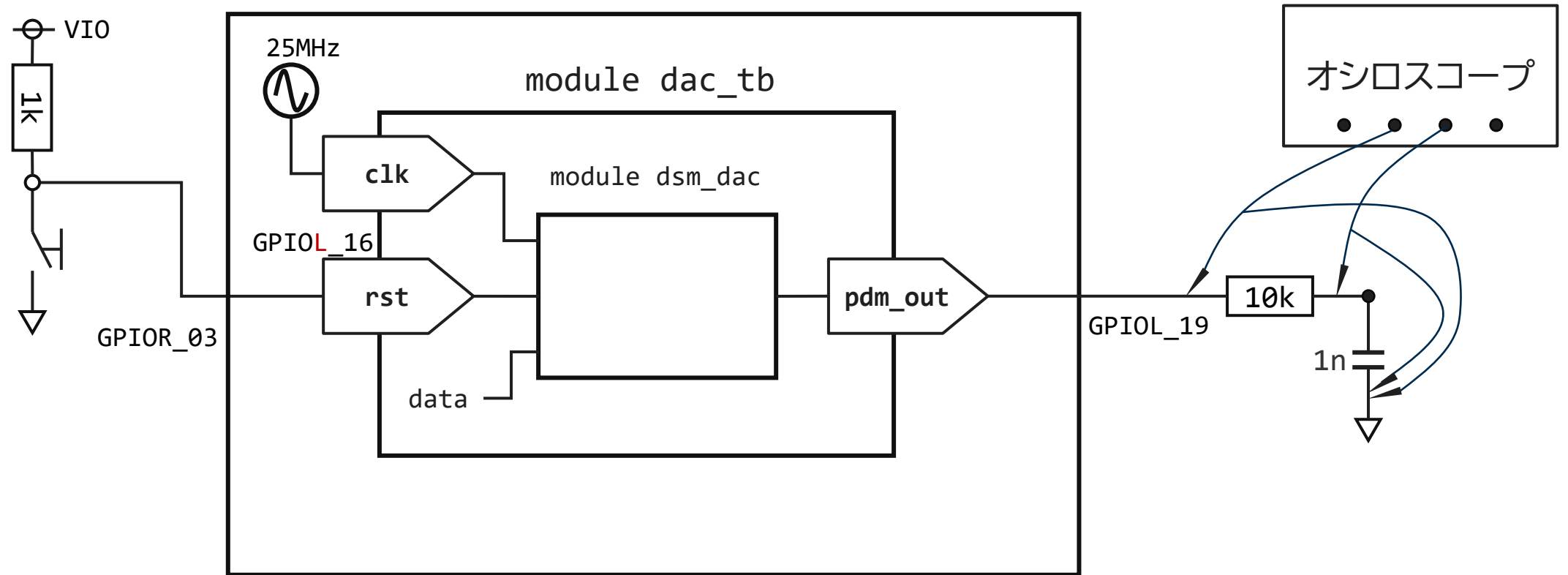
デジタル入力の $\Delta\Sigma$ 変調器5

これまでのブロックをまとめてみる



これをVerilogで書いてFPGAに書き込むことで、パルス密度で変調された信号が出力される→ローパスフィルタで高周波を落とすとアナログ信号が得られる

$\Delta \Sigma$ -DACの実験回路



$\Delta\Sigma$ -DACのVerilogコード

```
1 module dsm_dac(  
2     input wire clk,  
3     input wire xrst,  
4     input wire [9:0] data,  
5     output reg dsm_out);  
6  
7     wire [11:0] delta_add;  
8     wire [11:0] delta_fb;  
9     reg [11:0] sigma;  
10  
11     assign delta_fb = {sigma[11], sigma[11], 10'd0};  
12     assign delta_add = data + delta_fb;  
13  
14     always @(posedge clk or negedge xrst)  
15     begin  
16         if (!xrst)  
17         begin  
18             sigma <= 12'd0;  
19         end  
20         else  
21         begin  
22             sigma <= sigma + delta_add;  
23         end  
24     end  
25  
26     always @(posedge clk or negedge xrst)  
27     begin  
28         if (!xrst)  
29             dsm_out <= 1'b0;  
30         else  
31             dsm_out <= sigma[11];  
32     end  
33  
34 endmodule
```

2値化した出力をフィードバックする部分

入力とフィードバックした信号の足し算(Δ)
(delta_fb[9:0]に代入しても良い)

Δ した信号の加算(Σ)

量子化器(2値化する処理)
 $\Delta\Sigma$ 変調した信号は1回ラッチしてから出力

$\Delta\Sigma$ -DACのテストベンチVerilogコード

```
1  module dac_tb(  
2      input wire clk,  
3      input wire xrst,  
4      output wire pdm_out);  
5  
6  dsm_dac uut(  
7      .clk(clk),  
8      .xrst(xrst),  
9      .data(10'd10),  
10     .dsm_out(pdm_out));  
11  
12  endmodule
```

ΔΣ-DACのインスタンス

ここの値を変えてフィルタ出力がDACとして動作するか確認する

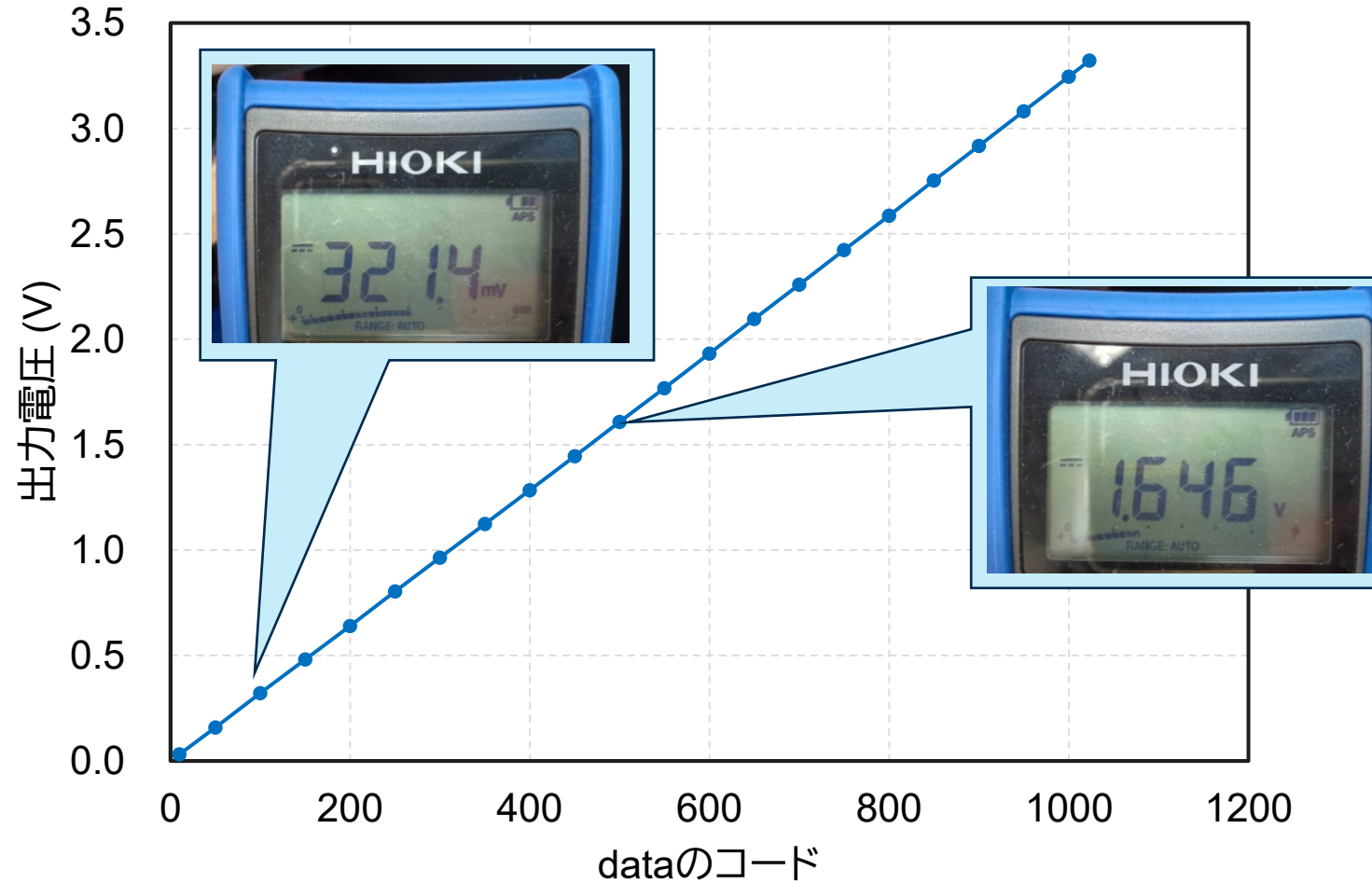
ピンアサイン

27

GPIO : Instance View

Instance	Package Pin	Resource	I/O Bank	Alt Conn	Features	Clock Region	Pad
clk	C2	GPIOL_16	1B	GCLK	None	L1	GPIOL_16_CLK2
pdm_out	G1	GPIOL_12	1A	GCTRL	None	L0	GPIOL_12_CTRL0
xrst	A6	GPIOR_03	2A	None	None	R1	GPIOR_03

$\Delta\Sigma$ -DACの出力電圧



出力電圧はコードに対して線形に増加 → DACとして動作している！

ノコギリ波を出力するテストベンチ

```
1 module dac_tb(  
2     input wire clk,  
3     input wire xrst,  
4     output wire pdm_out);  
5  
6 dsm_dac uut(  
7     .clk(clk),  
8     .xrst(xrst),  
9     .data(data),  
10    .dsm_out(pdm_out));  
11  
12 reg [9:0] data;  
13 reg [9:0] counter;  
14 always @(posedge clk or negedge xrst)  
15 begin  
16     if (!xrst)  
17     begin  
18         counter <= 10'd0;  
19         data <= 10'd0;  
20     end  
21     else  
22     begin  
23         counter <= counter + 10'd1;  
24         if (counter == 10'd1023) data <= data + 10'd1;  
25     end  
26 end  
27  
28 endmodule  
29
```

dataを更新するためのカウンタと
dataを更新するalways文
dataは1ずつ加算されて、オーバーフローすると、
ゼロに戻って加算され続ける

カウンタが1023になったときにdataを更新する

ノコギリ波の実験結果



フィルタ出力にはノコギリ波が出力されていることがわかる

まとめ

- CICフィルタを実装して2値の $\Delta\Sigma$ 変調された信号から多値の値に変換する処理を書きました。
 - いわゆるADCとして動作するまでができました。
- $\Delta\Sigma$ 変調を用いてデジタル信号からアナログ信号への変換を書きました。
 - いわゆるDACとして動作するまでができました。
- 次回はこのADCとDACを使って何か作ります。